BINV2150-A — Javascript : avancé

Séance 17 – Async/Await & Événements temporels

Haute École Léonard de Vinci

setTimeout

- Fonction JS permettant d'exécuter quelque chose après un certain temps
- Prends en argument :
  1. La fonction à exécuter
  2. Le nombre de millisecondes à attendre avant de l'exécuter
- Renvoie un nombre représentant un ID de l'événement temporel créé
- On peut annuler un événement temporel avec la fonction `clearTimeout`

```
const timer = setTimeout(() ⇒ {
    console.log("Hello!");
}, 1000);

if (cancel) clearTimeout(timer);
```

setInterval

- Fonction JS permettant de répéter quelque chose tous les certains temps
- Prends en argument :
  1. La fonction à exécuter
  2. Le nombre de millisecondes entre deux exécutions
- Renvoie un nombre représentant un ID de l'événement temporel créé
- On peut annuler l'interval avec la fonction `clearInterval`

```
const timer = setInterval(() ⇒ {
    console.log("Hello!");
}, 1000);

if (cancel) clearInterval(timer);
```

Exemple complet

```
const Slideshow = ({ photos }: SlideshowProps) => {
    const [selectedPhotoIndex, setSelectedPhotoIndex] = useState(0);

    const previousPhoto = () => { ... }
    const nextPhoto = () => { ... }

    useEffect(() => {
        const interval = setInterval(() => nextPhoto(), 3000);
        return () => clearInterval(interval);
    }, []);

    return (
        <>
            <Photo photo={photos[selectedPhotoIndex]} />
            <div>
                <button onClick={previousPhoto}>Previous Photo</button>
                <button onClick={nextPhoto}>Next Photo</button>
            </div>
        </>
    );
}
```

Async/Await

- Fonctions asynchrones renvoient des promesses
- On enregistre des callback avec `.then()` et `.catch()` pour traiter le résultat de la promesse

- Nouveaux mots clés pour écrire du code comme si tout était synchrone
- Une fonction notée `async` est une fonction asynchrone et renvoie une promesse
- `await fonctionAsynchrone()` bloque la fonction jusqu'à ce que la fonction asynchrone se termine
- On ne peut utiliser `await` que dans une fonction `async`
- Code équivalent mais plus facile à écrire

Comparaison

```
const fetchPizzas = () => {
    fetch("http://localhost:3000/pizzas")
        .then((response) => {
            if (!response.ok) throw new Error(`fetch error : ${response.status} : ${response.statusText}`);
            return response.json();
        })
        .then((pizzas: Pizza[]) => setPizzas(pizzas))
        .catch((err) => console.error("HomePage::error: ", err));
}
```

```
const fetchPizzas = async () => {
    try {
        const response = await fetch("http://localhost:3000/pizzas");
        if (!response.ok) throw new Error(`fetch error : ${response.status} : ${response.statusText}`);
        const pizzas: Pizza[] = await response.json();
        setPizzas(pizzas);
    } catch (err) {
        console.error("HomePage::error: ", err);
    }
}
```